

Missionaries and Cannibals Problem Using Python

Aim

To write a Python program to solve the **Crypt Arithmetic Problem** by assigning unique digits to letters such that the given arithmetic equation is satisfied.

Algorithm

1. Start the program.
2. Define the initial state (3, 3, 1) and the goal state (0, 0, 0).
3. Create a queue to store the states and a set to store the visited states.
4. Remove the front state from the queue.
5. If the current state is the goal state, display the solution and stop.
6. Generate all possible boat moves.
7. Check whether the generated state is valid and safe.
8. If the state is safe and not visited, add it to the queue.
9. Repeat Steps 4–8 until the goal state is reached.
10. Stop the program.

Python Code

```
from collections import deque

start = (3, 3, 1)

goal = (0, 0, 0)

def safe(m, c):
    return (m == 0 or m >= c) and (3 - m
    == 0 or 3 - m >= 3 - c)

def solve():
    q = deque([(start, [])])
    visited = set()
    while q:
        (m, c, b), path = q.popleft()
        if (m, c, b) == goal:
            return path + [(m, c, b)]
        if (m, c, b) in visited:
```

```

        continue

    visited.add((m, c, b))

    moves = [(1,0), (2,0), (0,1),
(0,2), (1,1)]

    for dm, dc in moves:

        if b == 1:

            nm, nc, nb = m - dm, c -
dc, 0

        else:

            nm, nc, nb = m + dm, c +
dc, 1

            if 0 <= nm <= 3 and 0 <= nc <=
3 and safe(nm, nc):

                q.append(((nm, nc, nb),
path + [(m, c, b)]))

    solution = solve()

    for step in solution:

        print(step)

```

EXP 5(Missionaries & Cannibals Problem).py > solve

```

1  from collections import deque
2  start = (3, 3, 1)
3  goal = (0, 0, 0)
4  def safe(m, c):
5      return (m == 0 or m >= c) and (3-m == 0 or 3-m >= 3-c)
6  def solve():
7      q = deque([(start, [])])
8      visited = set()
9      while q:
10         (m, c, b), path = q.popleft()
11         if (m, c, b) == goal:
12             return path + [(m, c, b)]
13         if (m, c, b) in visited:
14             continue
15         visited.add((m, c, b))
16         moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]
17         for dm, dc in moves:
18             if b == 1:
19                 nm, nc, nb = m-dm, c-dc, 0
20             else:
21                 nm, nc, nb = m+dm, c+dc, 1
22
23             if 0 <= nm <= 3 and 0 <= nc <= 3 and safe(nm, nc):
24                 q.append(((nm, nc, nb), path+[(m, c, b)]))
25
26 solution = solve()
27 for step in solution:
    print(step)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

(3, 3, 1)
(3, 1, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(1, 1, 1)
(0, 0, 0)

```